

Vorbereitungsklausur Programmierung II

Daniel Appenmaier, 27. Juli 2026

Aufgabe	Maximale Punktzahl	Erreichte Punktzahl
1	19	
2	17	
3	16	
Gesamt	52	

Hinweise

- Die in dieser Klausur verwendeten Personenbezeichnungen beziehen sich – sofern nicht anders kenntlich gemacht – auf alle Geschlechter
- Pakete und Klassenimporte müssen nicht angegeben werden
- Die Klassen und Annotationen der Bibliotheken *Lombok*, *JUnit 5* und *Mockito* dürfen verwendet werden
- Es kann davon ausgegangen werden, dass sämtliche Klassen entsprechende Implementierungen der Object-Methoden besitzen
- So nicht anders angegeben sollen Konstruktoren, Setter, Getter sowie die Object-Methoden wie in der Vorlesung gezeigt implementiert werden
- Die Konsolenausgaben-Methoden der Klasse `PrintStream` dürfen sinnvoll gekürzt geschrieben werden (zum Beispiel `sysout(„Hello World“)` statt `System.out.println(„Hello World“)`)
- Methoden- und Attributsbezeichner dürfen sinnvoll gekürzt geschrieben werden (zum Beispiel `getLWMCP()` statt `getLectureWithMostCreditPoints()`)

Wörterbuch

Englisch	Deutsch	Englisch	Deutsch
band name	Bandname	performance	Auftritt
description	Beschreibung	schedule	Spielplan
duplicate	Duplikat	stage	Bühne
festival	Festival	stage size	Bühnengröße
genre	Genre	start time	Startzeit
most used	am häufigsten genutzt	total	Gesamt

Aufgabe 1 (19 Punkte)

Erstelle die Klassen `Performance` (4 Punkte) und `Festival` (15 Punkte) anhand des abgebildeten Klassendiagramms.

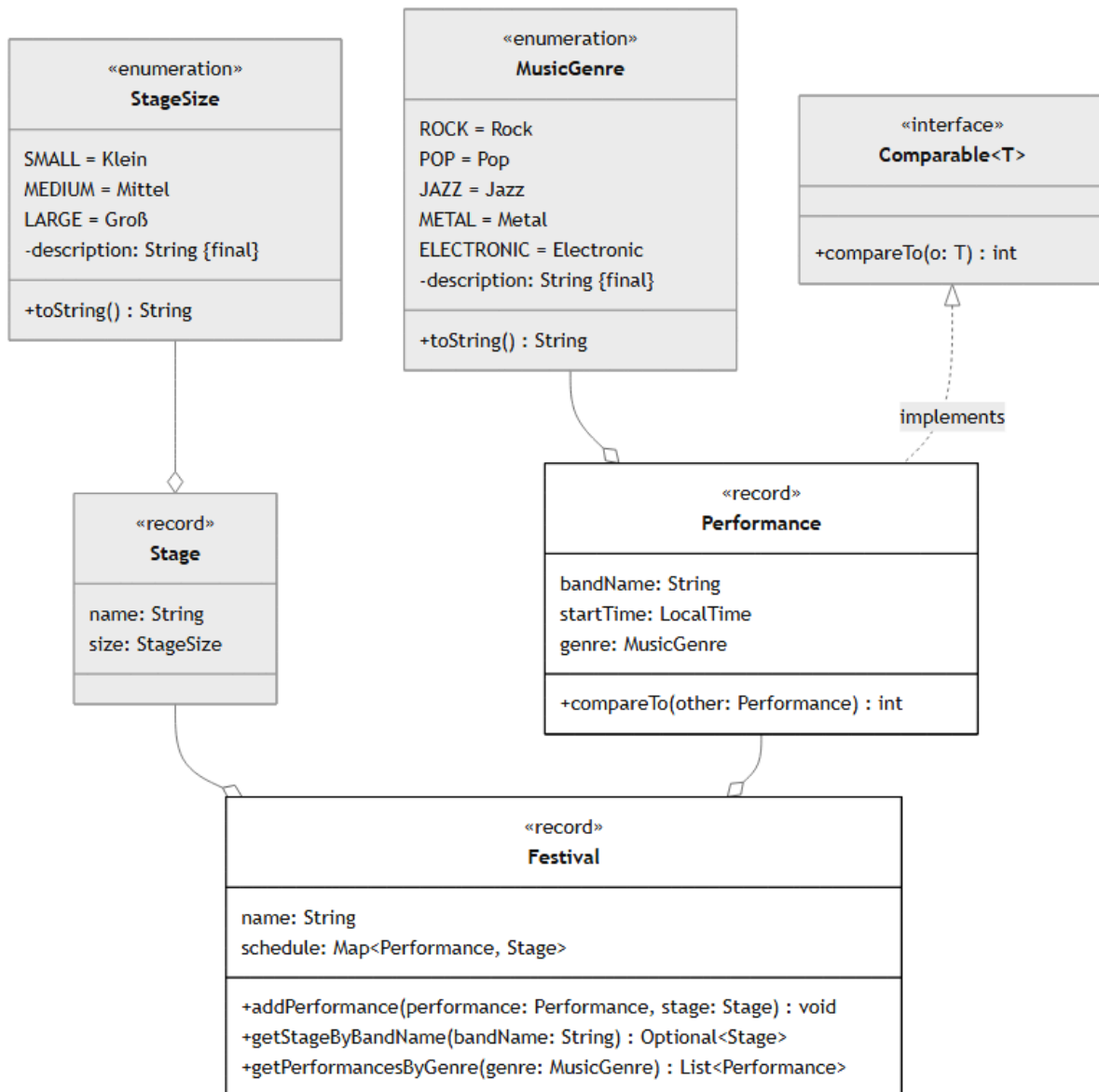


Abbildung 1: Klassendiagramm Aufgabe 1

Hinweise zur Klasse *Performance*

Die Methode `int compareTo(other: Performance)` soll so implementiert werden, dass Auftritte aufsteigend nach ihrer Startzeit sortiert werden können (2 Punkte).

Hinweise zur Klasse *Festival*

- Die Schlüssel-Werte-Paare des Assoziativspeichers `schedule` beinhalten als Schlüssel den Auftritt sowie als Wert die dazugehörige Bühne

- Die Methode `void addPerformance(performance: Performance, stage: Stage) throws DuplicatePerformanceException` soll einen neuen Schlüssel-Werte-Eintrag im Spielplan anlegen. Existiert der Auftritt bereits im Spielplan, soll die Ausnahme `DuplicatePerformanceException` ausgelöst werden (4 Punkte)
- Die Methode `Optional<Stage> getStageByBandName(bandName: String)` soll die Bühne zum eingehenden Bandnamen zurückgeben (5,5 Punkte)
- Die Methode `List<Performance> getPerformancesByGenre(genre: MusicGenre)` soll alle Auftritte des eingehenden Genres aufsteigend sortiert nach der Startzeit zurückgeben (4,5 Punkte)

Aufgabe 2 (17 Punkte)

Erstelle die JUnit-5-Testklasse `FestivalTest` anhand des abgebildeten Klassendiagramms.

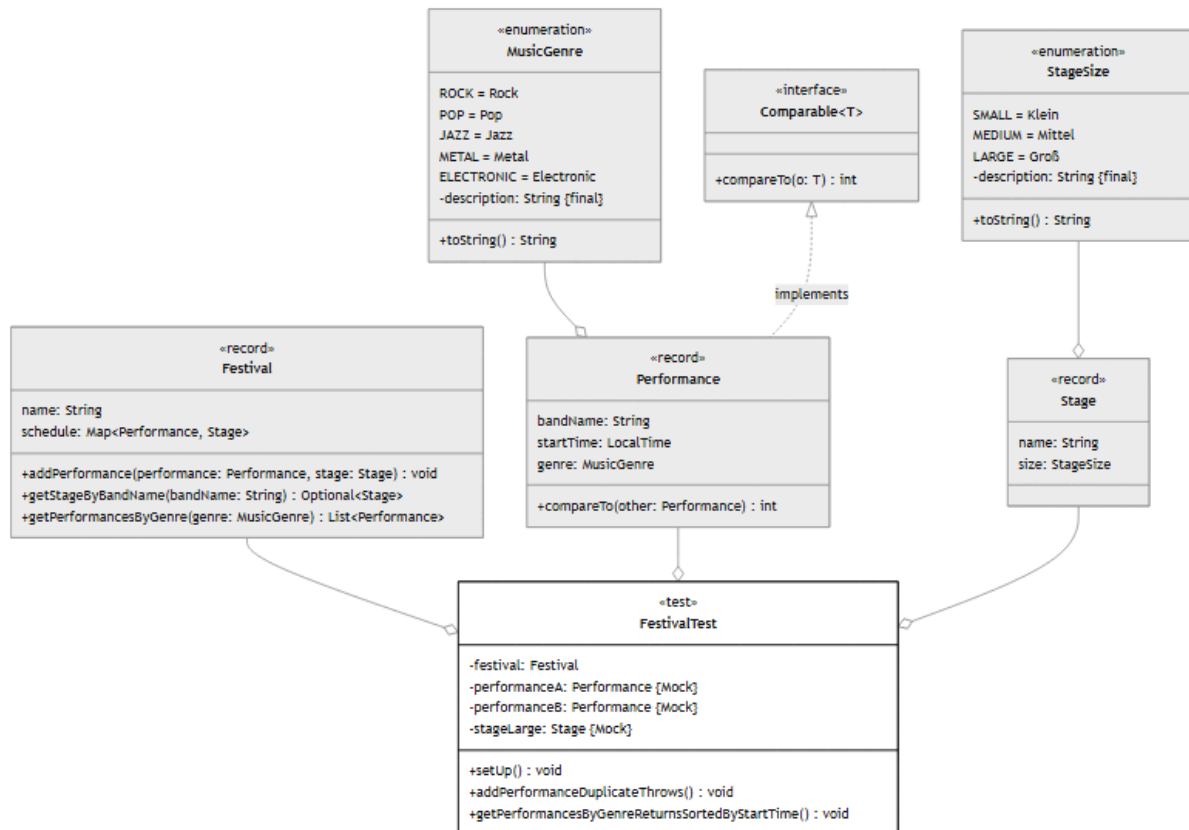


Abbildung 2: Klassendiagramm Aufgabe 2

Hinweise zur Klasse *FestivalTest*

- Die Lebenszyklus-Methode `void setUp()` soll alle Mock-Objekte über `MockitoAnnotations.openMocks(this)` initialisieren, ein `Festival` erstellen und dieses dem entsprechenden Attribut zuweisen. Anschließend soll der Auftritt `performanceA` mit der Bühne `stageLarge` sowie der Auftritt `performanceB` mit der Bühne `stageLarge` zum `Festival` hinzugefügt werden. Außerdem sollen die Mock-Objekte so konfiguriert werden, dass `performanceA` die Startzeit `12:00 Uhr` und das Genre `ROCK` sowie `performanceB` das Genre `POP` und die Startzeit `09:00 Uhr` zurückgibt (8 Punkte)
- Die Testmethode `void addPerformanceDuplicateThrows()` soll prüfen, ob beim erneuten Hinzufügen des bereits vorhandenen Auftritts `performanceA` die Ausnahme `DuplicatePerformanceException` ausgelöst wird (2,5 Punkte)
- Die Testmethode `void getPerformancesByGenreReturnsSortedByStartTime()` soll prüfen, ob beim Aufruf der Methode `List<Performance> getPerformancesByGenre(genre: MusicGenre)` mit dem Genre `ROCK` eine Liste mit einem Eintrag zurückgegeben wird, wobei `performanceA` an erster Stelle steht (4 Punkte)

Aufgabe 3 (16 Punkte)

Erstelle die Klasse `FestivalQueries` anhand des abgebildeten Klassendiagramms.

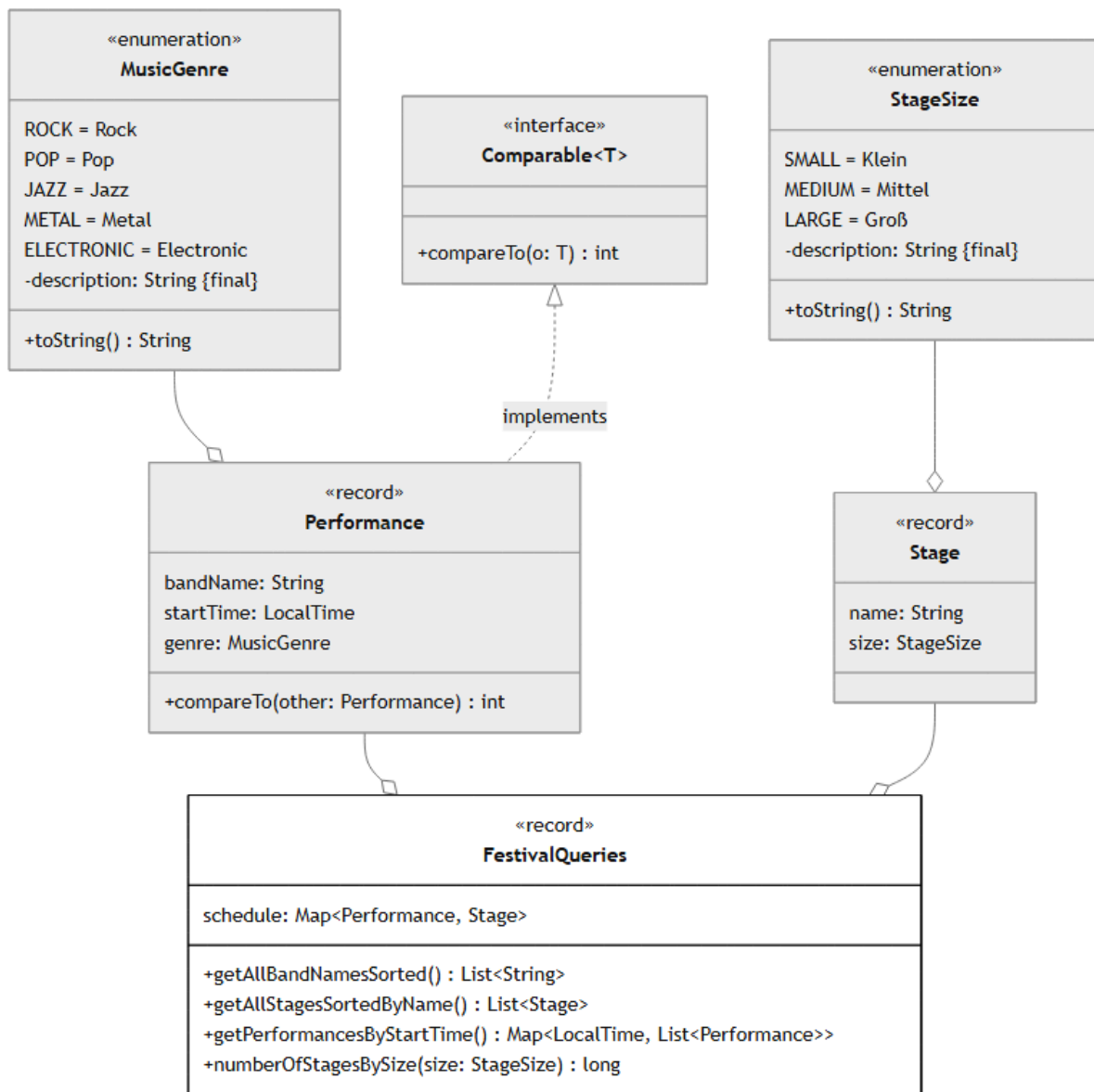


Abbildung 3: Klassendiagramm Aufgabe 3

Hinweise zur Klasse *FestivalQueries*

- Die Methode `List<String> getAllBandNamesSorted()` soll die Bandnamen aller Auftritte ohne Doppelungen aufsteigend sortiert zurückgeben (4 Punkte)
- Die Methode `List<Stage> getAllStagesSortedByName()` soll alle Bühnen ohne Doppelungen aufsteigend sortiert nach ihrem Namen zurückgeben (4 Punkte)
- Die Methode `Map<LocalTime, List<Performance>> getPerformancesByStartTime()` soll alle Auftritte gruppiert nach ihrer Startzeit zurückgeben (3,5 Punkte)
- Die Methode `long numberOfStagesBySize(size: StageSize)` soll die Anzahl der Einträge im Spielplan zurückgeben, bei denen die Bühne die eingehende Bühnengröße besitzt (3,5 Punkte)

Cheatsheet

java.lang

```

Boolean - int compare(x: boolean, y: boolean) //static
Boolean - Boolean valueOf(s: String) //static
Boolean - Boolean valueOf(b: boolean) //static
Comparable<T> - int compareTo(o: T)
Double - int compare(d1: double, d2: double) //static
Double - Double valueOf(s: String) //static
Double - Double valueOf(d: double) //static
Integer - int compare(x: int, y: int) //static
Integer - Integer valueOf(s: String) //static
Integer - Integer valueOf(i: int) //static
Long - long compare(x: long, y: long) //static
Long - Long valueOf(s: String) //static
Long - Long valueOf(l: long) //static
Object - boolean equals(object: Object)
Object - int hashCode()
Object - String toString()
Runnable - void run()
String - char charAt(index: int)
String - int length()
String - String[] split(regex: String)
String - String toLowerCase()
String - String toUpperCase()
System - long currentTimeMillis() //static

```

java.time

```

LocalDate - int getDayOfMonth()
LocalDate - int getDayOfYear()
LocalDate - Month getMonth()
LocalDate - int getMonthValue()
LocalDate - int getYear()
LocalDate - LocalDate now() //static
LocalDate - LocalDate of(year: int, month: int, dayOfMonth: int) //static
LocalDate - LocalDate parse(text: CharSequence) //static
LocalTime - int getHour()
LocalTime - int getMinute()
LocalTime - int getSecond()
LocalTime - LocalTime now() //static
LocalTime - LocalTime of(hour: int, minute: int, second: int) //static
LocalTime - LocalTime parse(text: CharSequence) //static

```

java.io

```

PrintStream - void print(obj: Object)
PrintStream - PrintStream printf(String format, Object... args)
PrintStream - void println()
PrintStream - void println(x: Object)

```

java.util

```

Collections - void sort(list: List<T>) //static
Collections - void sort(list: List<T>, c: Comparator<T>) //static
Comparator<T> - int compare(o1: T, o2: T)
Comparator<T> - Comparator<T> comparing(keyExtractor: Function<T,U>) //static
Entry<K,V> - K getKey()
Entry<K,V> - V getValue()
Entry<K,V> - V setValue(value: V)
Enumeration - Enumeration valueOf(arg0: String) //static
Enumeration - Enumeration[] values() //static
List<E> - boolean add(e: E)
List<E> - void add(index: int, element: E)
List<E> - boolean contains(o: Object)
List<E> - void forEach(action: Consumer<T>)
List<E> - E get(index: int)
List<E> - List<E> of(elements: E...) //static
List<E> - E remove(index: int)
List<E> - boolean remove(o: Object)
List<E> - List<T> reversed()
List<E> - int size()
List<E> - void sort(c: Comparator<T>)
Map<K,V> - boolean containsKey(key: Object)
Map<K,V> - boolean containsValue(value: Object)
Map<K,V> - Set<Entry<K,V>> entrySet()
Map<K,V> - void forEach(action: BiConsumer<K,V>)
Map<K,V> - V get(key: Object)
Map<K,V> - Set<K> keySet()
Map<K,V> - V put(key: K, value: V)
Map<K,V> - V putIfAbsent(key: K, value: V)
Map<K,V> - Collection<V> values()
Optional<T> - Optional<T> empty() //static
Optional<T> - T get()
Optional<T> - void ifPresent(action: Consumer<T>)
Optional<T> - void ifPresentOrElse(action: Consumer<T>, emptyAction: Runnable)
Optional<T> - boolean isPresent()
Optional<T> - Optional<T> of(t: T) //static
Optional<T> - Optional<T> ofNullable(t: T) //static
Optional<T> - T orElse(other: T)
OptionalDouble - OptionalDouble empty() //static
OptionalDouble - double getAsDouble()
OptionalDouble - void ifPresent(action: DoubleConsumer)
OptionalDouble - void ifPresentOrElse(action: DoubleConsumer, emptyAction: Runnable)
OptionalDouble - boolean isPresent()
OptionalDouble - OptionalDouble of(value: double) //static
OptionalDouble - double orElse(other: double)
Random - int nextInt(origin: int, bound: int)

```


java.util.function

```
BiConsumer - void accept(t: T, u: U)
Consumer<T> - void accept(t: T)
DoubleConsumer - void accept(value: double)
Function<T,R> - R apply(t: T)
Predicate<T> - boolean test(t: T)
ToDoubleFunction<T,R> - double applyAsDouble(value: T)
ToIntFunction<T,R> - int applyAsInt(value: T)
```

java.util.stream

```
Collectors - Collector<T,?,Map<K,List<T>>> groupingBy(classifier: Function<T,K>) //static
Collectors - Collector<CharSequence,?,String> joining(delimiter: CharSequence) //static
Collectors - Collector<T,?,Map<Boolean,List<T>>> partitioningBy(predicate:
Predicate<T>) //static
Collectors - Collector<T,?,Map<K,U>> toMap(keyMapper: Function<T,K>, valueMapper:
Function<T,U>) //static
Collectors - Collector<T,?,Set<T>> toSet() //static
DoubleStream - OptionalDouble average()
DoubleStream - double sum()
IntStream - OptionalDouble average()
IntStream - int sum()
Stream<T> - boolean allMatch(predicate: Predicate<T>)
Stream<T> - boolean anyMatch(predicate: Predicate<T>)
Stream<T> - R collect(collector: Collector<T,A,R>)
Stream<T> - long count()
Stream<T> - Stream<T> distinct()
Stream<T> - Stream<T> filter(predicate: Predicate<T>)
Stream<T> - Optional<T> findAny()
Stream<T> - Optional<T> findFirst()
Stream<T> - void forEach(action: Consumer<T>)
Stream<T> - Stream<T> limit(maxSize: long)
Stream<T> - Stream<R> map(mapper: Function<T,R>)
Stream<T> - DoubleStream mapToDouble(mapper: ToDoubleFunction<T,R>)
Stream<T> - IntStream mapToInt(mapper: ToIntFunction<T,R>)
Stream<T> - Optional<T> max(comparator: Comparator<T>)
Stream<T> - Optional<T> min(comparator: Comparator<T>)
Stream<T> - boolean noneMatch(predicate: Predicate<T>)
Stream<T> - Stream<T> skip(n: long)
Stream<T> - Stream<T> sorted()
Stream<T> - Stream<T> sorted(comparator: Comparator<T>)
Stream<T> - List<T> toList()
```

org.junit.jupiter.api

```
Assertions - void assertEquals(expected: Object, actual: Object) //static
Assertions - void assertFalse(condition: boolean) //static
Assertions - void assertNotEquals(expected: Object, actual: Object) //static
Assertions - void assertNotNull(actual: Object) //static
Assertions - void assertNotSame(expected: Object, actual: Object) //static
Assertions - void assertNull(actual: Object) //static
Assertions - void assertSame(expected: Object, actual: Object) //static
Assertions - T assertThrows(expectedType: Class<T>, executable: Executable) //static
Assertions - void assertTrue(condition: boolean) //static
Executable - void execute()
```

org.mockito / org.mockito.stubbing

```
Mockito - OngoingStubbing<T> when(methodCall: T) //static
MockitoAnnotations - AutoCloseable openMocks(testClass: Object) //static
OngoingStubbing<T> - OngoingStubbing<T> thenReturn(value: T)
```